04_info_2604032330

Informatique
(hors Python 3)

En marge de l'apprentissage de Python 3, ce document traite de notions théoriques de l'informatique. Il se veut le complément du support dédié à la programmation, afin de couvrir l'ensemble de l'exigible.

Jean Decroocq

CPGE TSI – 2025–2027
Lycée polyvalent de Cachan

Dernière mise à jour : 03/04/2026
Communiquer une anomalie : `jeandecroocq [at] hotmail.com`

Table des matières

1 Intégration numérique **3**
1.1 /// 3

2 Représentation des nombres en mémoire **4**
2.1 Bases de numération 4
2.2 Représentation des entiers 4
2.3 Représentation des nombres réels 5
2.4 Représentation des chaînes de caractères 6

1 Intégration numérique

1.1 ///

– ///

2 Représentation des nombres en mémoire

2.1 Bases de numération

- Le système décimal (base 10) est la norme de référence courante, mais l'architecture matérielle des ordinateurs repose sur le système binaire (base 2), dont les uniques symboles sont 0 et 1.
- L'écriture d'un nombre entier naturel dans une base B se décompose polynomialement. Un nombre représenté par la séquence $a_n a_{n-1} \dots a_1 a_0$ en base B possède une valeur décimale équivalente définie par
$$\sum_{k=0}^{n} a_k B^k,$$
où chaque chiffre a_k appartient à l'intervalle d'entiers $[0, B-1]$.
- Le système hexadécimal (base 16) est privilégié en électronique numérique car il permet une représentation très compacte des séquences binaires. Il utilise les chiffres de 0 à 9, complétés par les lettres de A à F (de 10 à 15).
- Le passage d'une base à une autre repose sur des méthodes algorithmiques :
  - D'une base B vers la base 10 : Évaluation du polynôme des puissances successives de la base. Par exemple, $BC5_{16} = 11 \times 16^2 + 12 \times 16^1 + 5 \times 16^0 = 3013_{10}$.
  - De la base 10 vers une base B : Divisions euclidiennes successives du quotient par la base B jusqu'à obtenir un quotient nul. La lecture des restes de bas en haut (du dernier au premier) donne l'écriture dans la nouvelle base.
  - Entre les bases 2 et 16 : Une conversion directe est possible en regroupant les bits par paquets de quatre (un quartet ou *nibble*). Par exemple, la lecture du motif $1100\,0000_2$ s'associe directement à $C0_{16}$.
- Le langage Python gère nativement ces différentes bases au moyen de préfixes et de fonctions de conversion spécifiques.

```python
# saisie directe via prefixe
valeur_bin = 0b1101  # base 2 (vaut 13)
valeur_hex = 0xA3    # base 16 (vaut 163)

# conversion d'un entier vers une chaine
chaine_bin = bin(13)  # renvoie '0b1101'
chaine_hex = hex(163) # renvoie '0xa3'
```

2.2 Représentation des entiers

- Les opérations au sein du microprocesseur s'effectuent sur des blocs d'information appelés *mots-machine*, dont la taille n s'exprime en bits (généralement 32 ou 64 bits). L'*octet* (8 bits) constitue l'unité usuelle de stockage en mémoire.
- Les entiers naturels sont encodés en binaire direct. Un mot de n bits permet de couvrir l'intervalle d'entiers allant de 0 à $2^n - 1$. Le dépassement de cette borne supérieure engendre une erreur matérielle par dépassement de capacité (*overflow*). Python, en revanche, gère dynamiquement la taille des entiers en s'affranchissant de cette limite matérielle.
- Le codage des entiers relatifs (signés) s'effectue par la méthode du *complément à 2*. L'intervalle de représentation est translaté pour s'étendre de -2^{n-1} à $2^{n-1} - 1$.

- Dans cette représentation, le bit de poids fort indique le signe de la valeur : 0 désigne un entier positif, et 1 un entier négatif.
- L'obtention du code d'un entier négatif a (où $a < 0$) sur n bits peut se faire par deux approches :
  - Approche mathématique : Coder en binaire direct la valeur positive résultant de l'opération $a + 2^n$.
  - Approche algorithmique : Écrire la valeur absolue de a en binaire, inverser tous les bits (les 0 deviennent 1, et inversement), puis ajouter la valeur 1 au résultat.

2.3 Représentation des nombres réels

- La conversion d'un nombre à virgule en binaire requiert un traitement séparé de sa partie entière et de sa partie fractionnaire.
- La méthode de conversion de la partie fractionnaire consiste à la multiplier successivement par 2. À chaque itération, la partie entière du résultat (nécessairement 0 ou 1) constitue le chiffre binaire suivant de la partie fractionnaire, de gauche à droite. Le processus se répète avec la nouvelle partie fractionnaire jusqu'à ce que celle-ci devienne nulle ou que la limite de la mémoire soit atteinte.
- Les ordinateurs utilisent la représentation en *virgule flottante* normalisée par l'IEEE 754. Cette norme attribue trois rôles distincts aux bits du mot-machine :
  - Le *signe* (s) : 1 bit (0 pour positif, 1 pour négatif).
  - L'*exposant* (e) : Décalé d'une valeur fixe (127 en simple précision sur 8 bits, 1023 en double précision sur 11 bits) pour éviter l'usage d'un exposant signé.
  - La *mantisse* (m) : Représente les chiffres significatifs de la partie fractionnaire.
- La conversion manuelle d'un réel (par exemple $-243,25$) vers la norme IEEE 754 s'opère ainsi :
  - Conversion en binaire complet : $-11110011,01_2$.
  - Décalage de la virgule (normalisation) : $-1,111001101_2 \times 2^7$. L'unique bit 1 avant la virgule est implicite et ne sera pas mémorisé.
  - Calcul de l'exposant décalé : $7 + 127 = 134 = 10000110_2$.
  - La mantisse est complétée par des zéros à droite pour atteindre la taille requise.
- L'évaluation de la valeur x à partir de ses champs (s, e, m) dépend du contenu de l'exposant :
  - *Nombres normalisés* : Si l'exposant n'est ni rempli de zéros ni de uns, la valeur est $x = (-1)^s \times (1, m_1 m_2 \dots) \times 2^{e-\text{décalage}}$.
  - *Nombres dénormalisés* : Si $e = 0$ et $m \neq 0$, le bit implicite devient 0, permettant de coder des nombres extrêmement proches de zéro.
  - *Zéro* : Si $e = 0$ et $m = 0$. Il existe ainsi un $+0$ et un -0.
  - *Infinis* : Si e est à sa valeur maximale (que des 1) et $m = 0$, le nombre représente $\pm\infty$.
  - *NaN (Not a Number)* : Si e est à sa valeur maximale et $m \neq 0$, signale une erreur ou une indétermination mathématique.

```python
# affichage de l'ecriture hexadecimale interne du flottant
valeur = 5.5
print(valeur.hex())
# affiche 0x1.6000000000000p+2
```

- La représentation fractionnaire binaire induit un inconvénient majeur : certains nombres décimaux simples, tels que $0,1$, possèdent un développement binaire infini et périodique. Ils ne peuvent donc pas être stockés de manière exacte et subissent une troncature matérielle.

- Ces approximations s'accumulent lors des opérations arithmétiques, entraînant des erreurs de précision notables. Par exemple, l'évaluation de `0.1 + 0.2` en Python renvoie `0.30000000000000004`.
- Il est par conséquent impératif de ne jamais vérifier l'égalité stricte entre deux nombres flottants avec l'opérateur `==`. La comparaison s'effectue en validant que la valeur absolue de leur différence demeure inférieure à une tolérance d'erreur infinitésimale.

2.4 Représentation des chaînes de caractères

- Les chaînes de caractères obéissent à une logique de représentation totalement différente. Elles nécessitent une table de correspondance liant un symbole à une séquence de bits.
- L'encodage historique ASCII associe 128 caractères à des codes sur 7 bits (stockés sur un octet). Pour pallier ses limites et intégrer l'intégralité des symboles internationaux, la norme moderne UTF-8 est utilisée, encodant les caractères sur une longueur variable d'octets.